Chasis robot de laberinto

Dos iteraciones sobre la misma base de 125 × 125 mm

	Iteración A · ATmega328P	Iteración B · Arduino UNO Q
Cerebro	Arduino UNO R3 (ATmega328P)	Arduino UNO Q (Linux + STM32)
Resuelve el laberinto	Flood-fill en el MCU	Flood-fill en Python (Linux)
Paredes	3 × VL53L0X	3 × VL53L0X + cámara (celda siguiente)
Marcas de color	TCS34725 (I2C)	TCS34725 (I2C)
Pantalla	OLED SSD1306 128×64	OLED SSD1306 128×64
Motores	2 × N20 sin encoder + L298N	2 × N20 sin encoder + L298N
Memoria del mapa	EEPROM interna	Archivo /app/mapa.json
Mástil de cámara	No	Sí, ajustable 150–200 mm
Altura total aprox.	80 mm	≈ 205 mm

✔ Huella máxima 125 × 125 mm incluyendo ruedas (lineamiento: ≤ 125 × 125 mm, altura libre).

✔ Esquinas redondeadas R30: radio de giro 76 mm < 90 mm (medio pasillo). Gira en su lugar sin rozar.

✔ Ruedas de 20 mm de ancho por lado (40 mm en total). Queda 85 mm de chasis entre ruedas.

✔ Sensor de color I2C, OLED, interruptor de arranque: incluidos (lineamientos de diseño).

Supuesto: rueda Ø34 mm (la típica de N20). Si la suya es distinta, cambie D_RUEDA en chasis.scad.

Las dos iteraciones usan las mismas piezas impresas; la B agrega el mástil y la cámara.

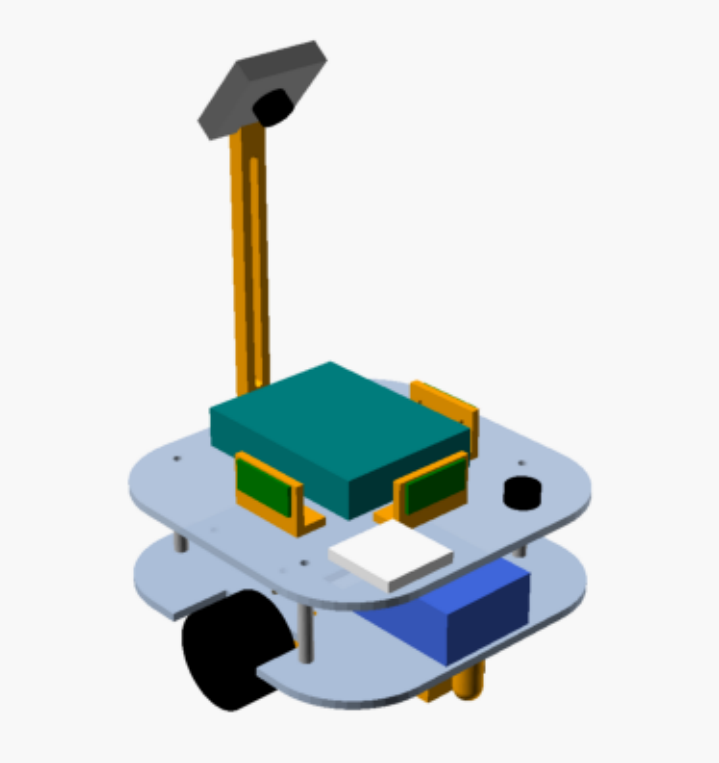
Vistas 3D

Renderizadas desde chasis.scad (OpenSCAD)

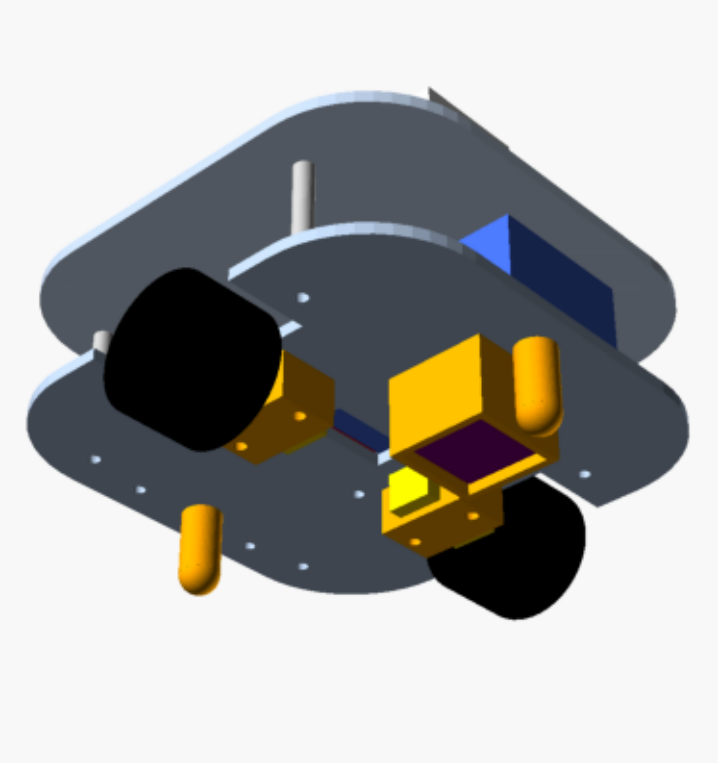
Iteración A · ATmega328P



Iteración B · UNO Q con mástil



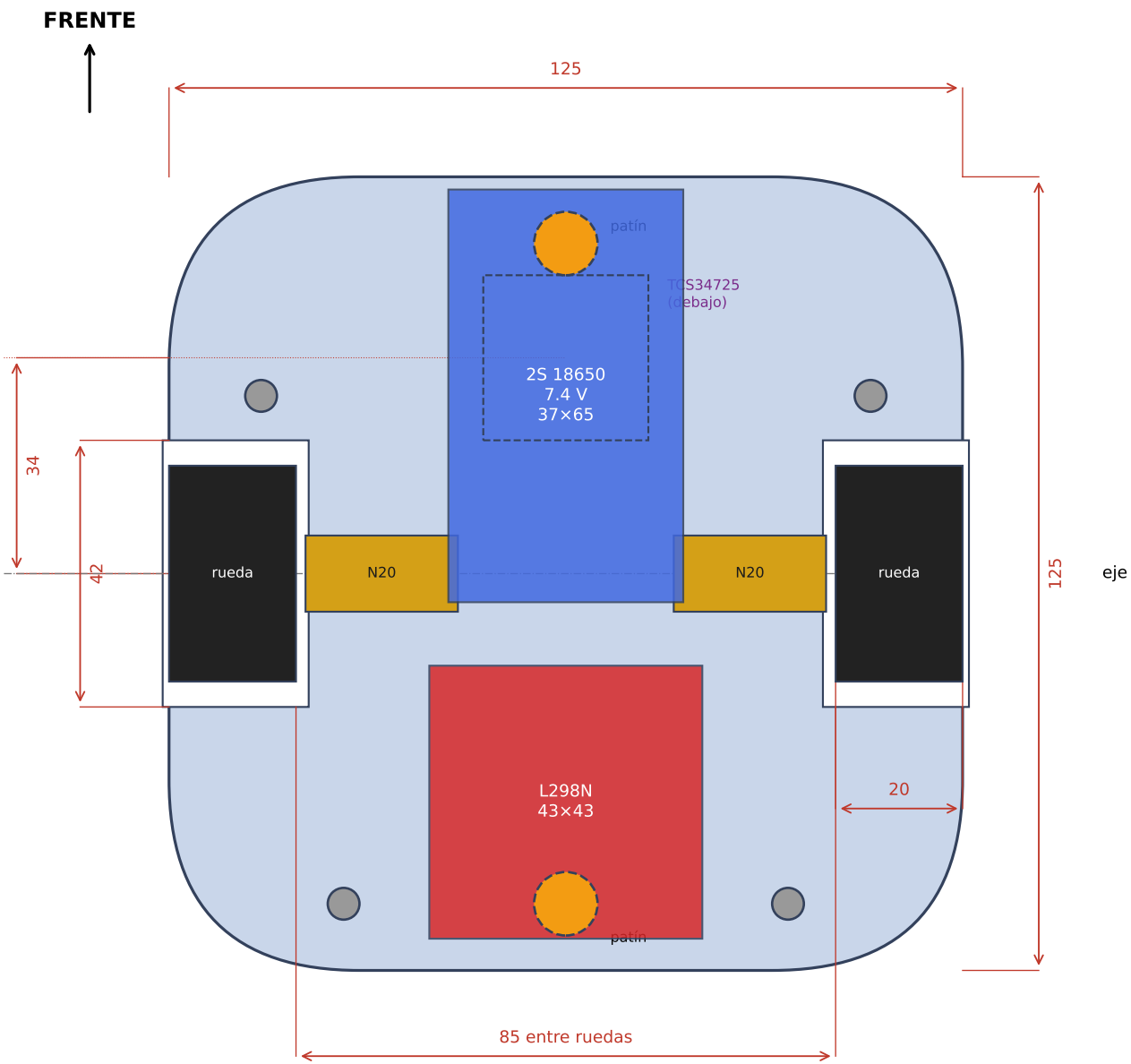
Vista inferior: patines, sensor de color, motores



Pisos impresos · Soportes impresos · Placa Arduino · VL53L0X · Batería 2S 18650 · L298N · OLED

Piso 1 · planta (vista superior)

Placa de 3 mm a 22 mm del suelo. Motores debajo, batería y L298N encima.



Debajo del piso 1

- 2 cunas impresas aprietan cada N20 contra el piso (M3).
- Eje a 17 mm del suelo (rueda Ø34).
- Capucha del sensor de color a 34 mm delante del eje; la cara del sensor queda a 4 mm del suelo y la capucha tapa la luz ambiente.
- 2 patines (delante y detrás, y = ±52) dejan 0.5 mm al suelo: el robot se apoya en las ruedas.

Encima del piso 1

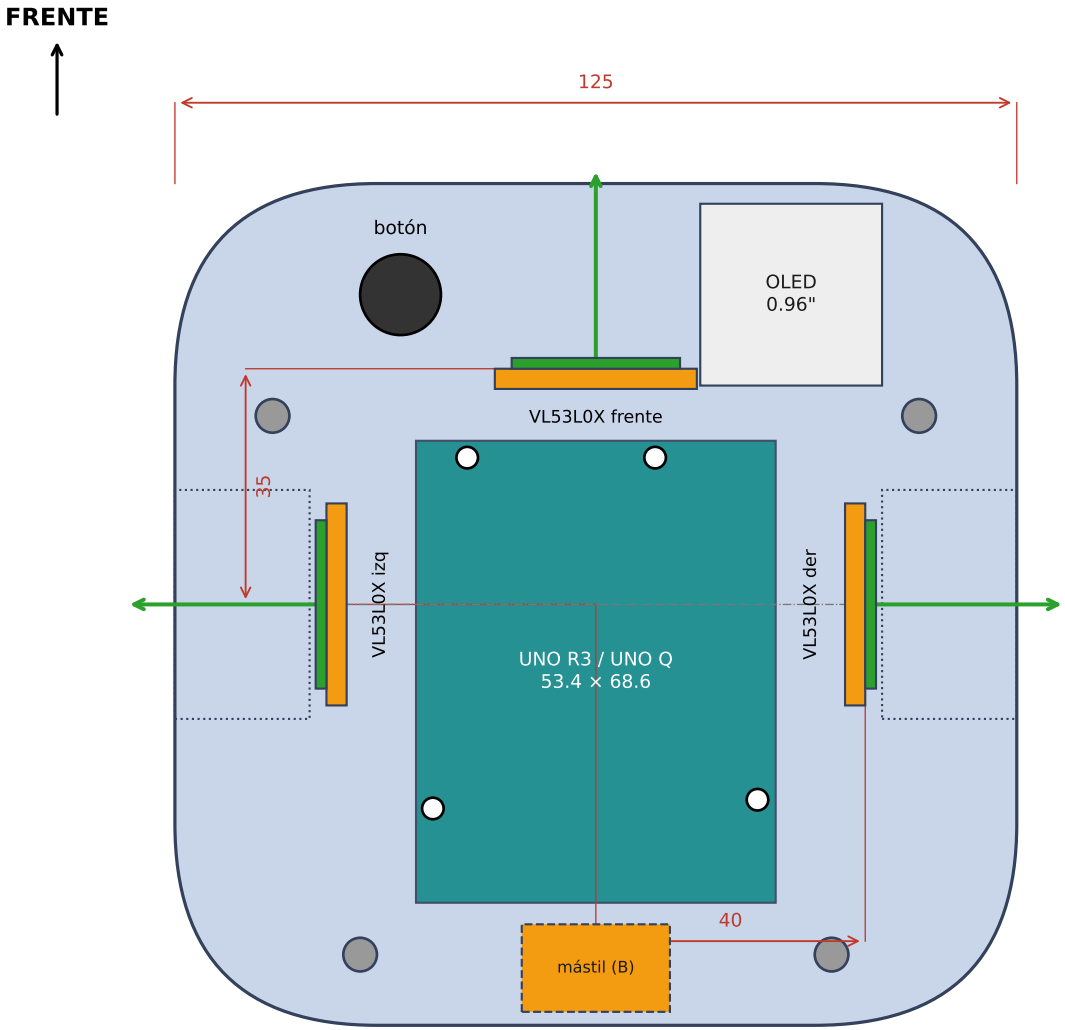
- Batería 2S 18650 al frente (peso bajo y adelante del eje para que no cabecee al frenar).
- L298N atrás; su disipador mide 27 mm y cabe bajo el piso 2 (hueco libre 32 mm).
- Ventanas al centro para pasar cables.
- 4 separadores M3 × 32 mm en (±48, 28) y (±35, −52).

Por qué así

- Eje al centro: gira sobre su propio lugar, lo que simplifica los giros de 90° por tiempo.
- OFF_COLOR del código = 34 / 180 ≈ 0.2 celdas.

Piso 2 · planta (vista superior)

Placa de 3 mm a 57 mm del suelo. Arduino, sensores de distancia, OLED, botón y mástil.



Sensores de distancia VL53L0X

- Van en el piso 2 porque las ruedas tapan los lados en el piso 1. Las paredes (≈10 cm) los cubren.
- Laterales a ±40 mm del centro, sobre el eje: leen la celda donde está el robot.
- Frontal a 35 mm delante del eje.
- Van metidos hacia dentro a propósito: así nunca leen a menos de 4 cm, donde el VL53L0X es impreciso.

Placa

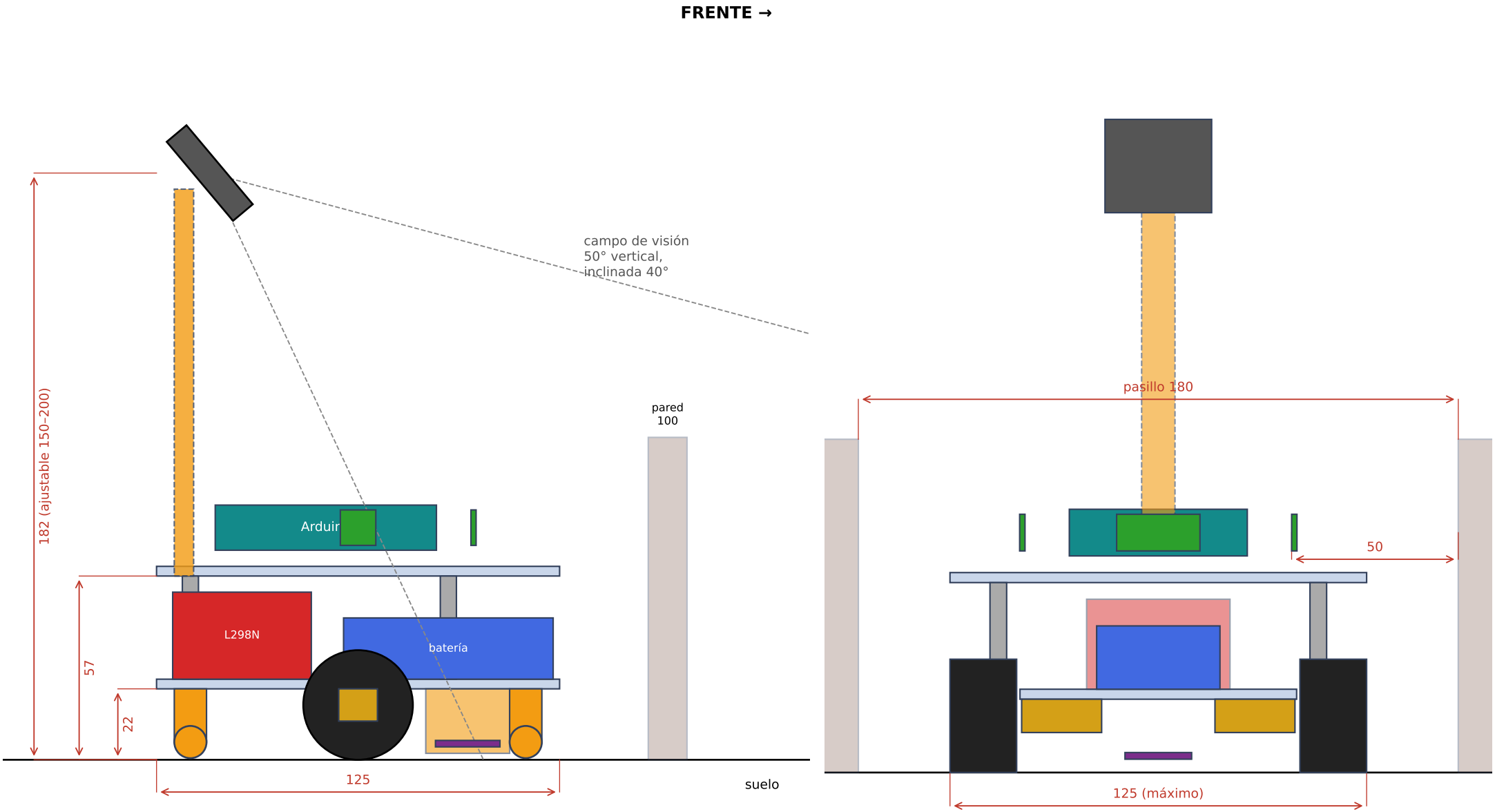
- UNO R3 y UNO Q tienen la misma huella y agujeros: el mismo piso 2 sirve para las dos iteraciones.
- Separadores de 5 mm bajo la placa.

Iteración B

- El mástil entra en la ranura trasera (10 × 6 mm).
- El hub USB-C de la cámara va en el piso 1, en la zona central entre la batería y el L298N.

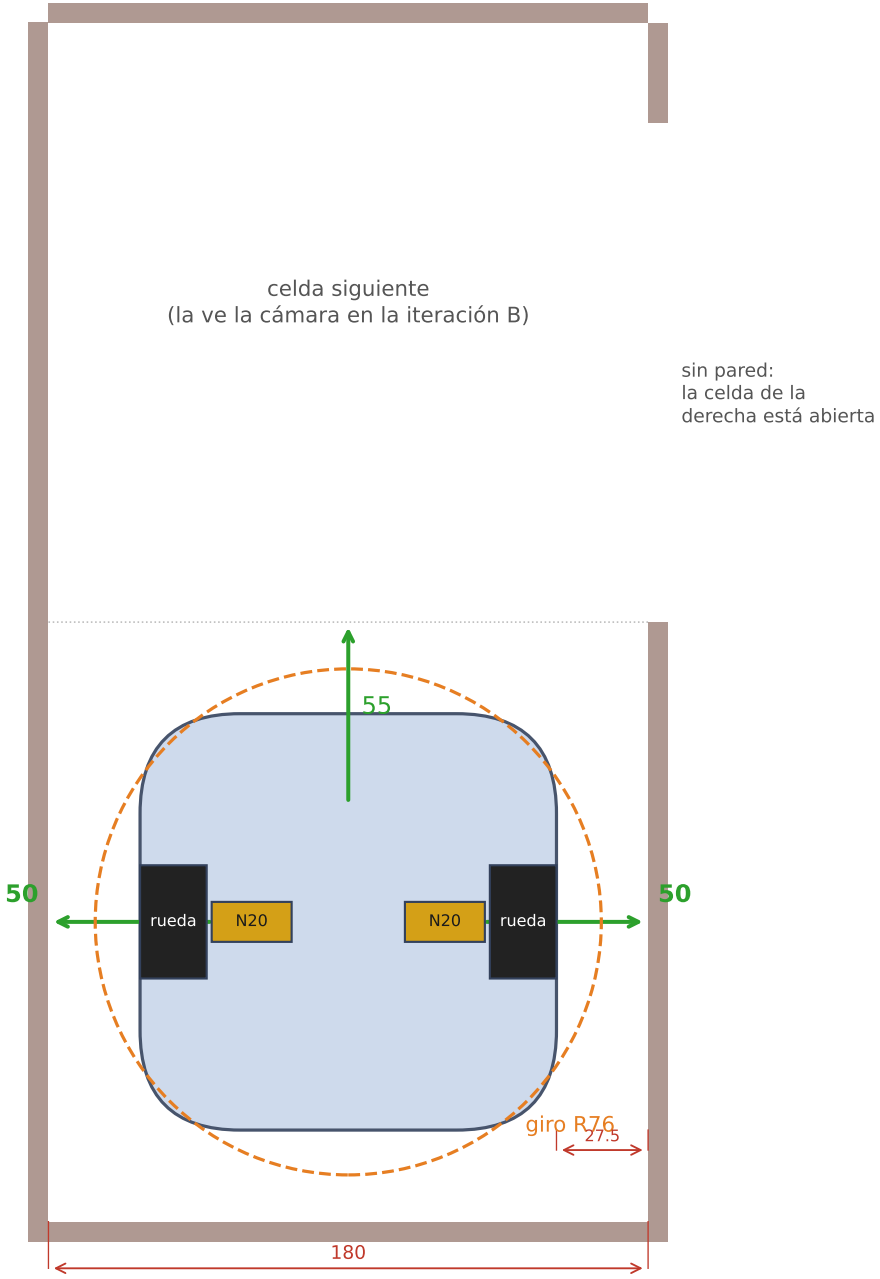
Alzados

Vista lateral (izquierda) y frontal (derecha). La cámara solo va en la iteración B.



El robot dentro del laberinto

Celda de 180 mm: holguras, lecturas esperadas de los sensores y radio de giro



Valores que usa el código

- D_LADO = 50 mm: lectura lateral con el robot centrado.
- D_FRENTE = 55 mm: lectura frontal al llegar al centro de una celda con pared enfrente. El robot frena por distancia en vez de por tiempo y corrige el error acumulado.
- PARED = 120 mm: si la lectura es menor, hay pared en esa celda; la siguiente pared quedaría a ≈230 mm.

Holguras

- 27.5 mm por lado con el robot centrado.
- Radio de giro 76 mm contra 90 mm de medio pasillo: sobran 14 mm al girar en su lugar.
- Sin las esquinas redondeadas el radio sería 88 mm y rozaría las paredes.

Calibración rápida

1. Centrar el robot a mano en una celda y anotar las 3 lecturas (modo calibración: encender con el botón presionado). Esos son D_LADO y D_FRENTE reales.
2. Ajustar MS/T_CELDA hasta que avance 180 mm en un pasillo sin pared enfrente.

Conexiones

Mismo cableado en las dos iteraciones salvo donde se indica

Tabla de pines

Componente	Pin del módulo	UNO R3 (A)	UNO Q (B)
L298N	ENA (sin jumper)	D5	D9
L298N	IN1 / IN2	D7 / D8	D4 / D5
L298N	ENB (sin jumper)	D6	D10
L298N	IN3 / IN4	D9 / D10	D7 / D8
L298N	12V / GND	Batería + / GND	Batería + / GND
L298N	OUT1-2 / OUT3-4	Motor izq / der	Motor izq / der
VL53L0X izq	XSHUT	A0	A0
VL53L0X frente	XSHUT	A1	A1
VL53L0X der	XSHUT	A2	A2
VL53L0X ×3	SDA / SCL	A4 / A5	SDA / SCL
TCS34725	SDA / SCL	A4 / A5	SDA / SCL
OLED SSD1306	SDA / SCL	A4 / A5	SDA / SCL
Sensores y OLED	VCC	5V	3.3V
Botón arranque	a GND	D2	D12
Batería 2S	+	VIN	VIN
Cámara USB	—	no lleva	hub USB-C

Direcciones I2C

- TCS34725: 0x29
- OLED: 0x3C
- VL53L0X: salen todos en 0x29 (igual que el sensor de color). El código los apaga con XSHUT y los prende uno por uno para darles 0x30, 0x31 y 0x32.

Alimentación

- GND común: batería, L298N, Arduino y sensores.
- No usar el regulador de 5 V del L298N para el Arduino.
- En la iteración B los pines son de 3.3 V. El L298N acepta esa lógica.

¿Por qué pines distintos en A?

- En el UNO R3, el Timer1 se usa como base de tiempo de 10 ms (requisito de temporizadores). Eso quita el PWM de D9/D10, así que los motores usan D5/D6 (Timer0).

Piezas a imprimir y lista de material

Piezas impresas

Pieza (PIEZA= en chasis.scad)	Cant.	Notas
piso1	1	PLA/PETG, 3 mm, 30% relleno
piso2	1	Igual
soporte_motor	2	Imprimir boca abajo
soporte_color	1	Negro mate (tapa la luz)
patin	2	Lijar la punta; ajustar altura
soporte_tof	3	Escuadra para el VL53L0X
mastil	1	Solo iteración B; acostado

Lista de material

Material	Cant.	A	B
Arduino UNO R3 (ATmega328P)	1	✔	
Arduino UNO Q	1		✔
Motorreductor N20 6 V (300–500 RPM)	2	✔	✔
Rueda N20 Ø34 × 20 mm	2	✔	✔
Puente H L298N (Steren)	1	✔	✔
Sensor de color TCS34725	1	✔	✔
Sensor de distancia VL53L0X	3	✔	✔
OLED SSD1306 0.96" I2C	1	✔	✔
Botón pulsador	1	✔	✔
Batería 2S 18650 7.4 V 3000 mAh (conector T)	1	✔	✔
Cargador balanceador 2S (JST-XH 3 pines)	1	✔	✔
Cámara USB + hub USB-C	1		✔
Separador M3 × 32 / tornillos M3	4 / 20	✔	✔

```
Exportar STL:  openscad -D 'PIEZA="piso1"' -o piso1.stl chasis.scad
```

Ver el ensamble: abrir chasis.scad en OpenSCAD y cambiar VERSION a "unoq" o "328p".